

PROJECT DOCUMENTATION REPORT

Olist E-Commerce Performance Dashboard

SQL Data Auditing · Power BI Data Modeling · DAX · Interactive Reporting

Prepared by

Mostafa Abdelhamed

Data Analyst

September 2026

mostafa22tito@gmail.com

(+2) 01022100347 Ousim, Giza, Egypt

[LinkedIn](#) | [GitHub](#)

Table of Contents

1. Executive Summary
2. Project Objectives
3. Dataset Overview
4. Methodology
 - 4.1 Data Exploration & Quality Assurance (SQL)
 - 4.2 Data Modeling
 - 4.3 DAX Measures Layer
 - 4.4 Report Design
5. Tools & Technologies
6. Data Quality Approach & Challenges
7. Skills Demonstrated
8. Conclusion
- A. Appendix — SQL Reference

1. Executive Summary

This report documents the end-to-end development of the Olist E-Commerce Performance Dashboard, an analytics project built on Olist's public Brazilian e-commerce order data. Olist is a marketplace platform that connects small merchants across Brazil to major online sales channels; its public dataset covers roughly 100,000 orders placed between 2016 and 2018.

The project follows a realistic analytics pipeline rather than jumping straight to visuals: raw relational tables were first explored and audited in SQL Server, then modeled and calculated in Power BI, and finally presented through a five-page interactive report addressing sales performance, logistics efficiency, and customer satisfaction.

The deliverables covered in this document are: the SQL data-preparation script, the Power BI data model and DAX measures layer, and the final Power BI report (olist.pbix).

2. Project Objectives

The dashboard was designed to answer the following business questions for a marketplace operations/management audience:

- How is revenue trending over time, and which states and product categories drive it?
- Are orders being delivered on time, and where are delays concentrated geographically?
- How does delivery performance relate to freight cost?

- How satisfied are customers overall, and which product categories generate the most negative reviews?
- Which payment methods and installment patterns do customers use most?

3. Dataset Overview

Source: Brazilian E-Commerce Public Dataset by Olist (Kaggle) — real, anonymized commercial data covering approximately 100,000 orders placed across multiple Brazilian marketplaces between 2016 and 2018.

Core tables used in this project:

- customers — customer ID and location (city/state)
- orders — order status and purchase/delivery/estimated-delivery timestamps
- order_items — line items per order, price, and freight value
- order_payments — payment type, installment count, and payment value
- order_reviews — customer review score per order
- products — product attributes, linked to category
- product_category_name_translation — maps category codes to English names
- sellers — seller ID and location

4. Methodology

4.1 Data Exploration & Quality Assurance (SQL)

Before any modeling took place, the raw tables were explored and validated directly in SQL Server (T-SQL). This phase was split into two parts.

Part 1 — Exploration:

- Row counts and distinct values (customer states, delivered vs. total orders)
- Top 10 highest-priced products
- Transaction counts by payment type
- Top seller cities by number of sellers
- Review score distribution
- Price and freight summary statistics (min / max / average)
- Orders ranked by number of payment installments

Part 2 — Data Quality Checks:

- Delivered orders with a missing delivery date
- Zero/negative item prices or negative freight values
- Discrepancies between item + freight totals and the actual amount paid
- Logically invalid dates (delivery date earlier than purchase or carrier hand-off)
- Average delivery time vs. the estimated delivery date, to quantify delay
- Revenue ranked by product category and by customer state

This phase also produced a reusable reporting view joining orders to customer location:

```
CREATE VIEW vw_orders_master AS
SELECT o.order_id, o.customer_id, o.order_status,
       o.order_purchase_timestamp, o.order_delivered_customer_date,
       o.order_estimated_delivery_date,
       c.customer_city, c.customer_state
FROM dbo.orders o
JOIN dbo.customers c ON o.customer_id = c.customer_id;
```

4.2 Data Modeling

The Power BI data model connects the core Olist entities in a star-schema-style layout, with orders at the center. Customers, order_items, order_payments, order_reviews, products (translated to English category names via product_category_name_translation), and sellers surround it as supporting tables, alongside a Date Hierarchy used for all time-based analysis.

4.3 DAX Measures Layer

A dedicated measures table holds 28 DAX measures, organized into three families that map directly to the report's three analytical pillars. The table below lists the measures referenced directly by the report's visuals:

Sales & Revenue	Logistics & Delivery	Customer Satisfaction
Total Revenue	AVG Delivery Days	Avg Review Score
Total Orders	AVG Estimated Delivery Days	Total Reviews
Total Customers	ON-Time Delivery Rate	Positive Reviews
Total Items	Total Freight	Negative Reviews
	Late Deliveries Count	Positive Review Rate
		Negative Review Rate

Note: the remaining measures in the 28-measure library support these headline calculations (e.g. underlying sums and counts) but are not surfaced directly on a visual.

4.4 Report Design

The final report spans five pages, each targeting a different stakeholder view:

Welcome Page

- Landing/navigation page with a page-navigator control and branded title
- Quick-glance teasers summarizing each section: logistics (avg. delivery days, on-time rate, freight vs. price), payment methods, top categories, and customer experience
- Total Revenue KPI card and a Monthly Revenue & Order Volume chart

Executive Overview

- Five KPI cards: Total Revenue, Total Orders, Total Customers, AVG Delivery Days, Avg Review Score
- Revenue trend line chart over time; review-score pie chart; payment-type donut chart
- Revenue by product category (bar chart) and by customer state (combo chart)
- Slicers: customer state, payment type, review score, year

Sales & Product

- KPI cards: Total Revenue, item count, Total Customers, AVG Delivery Days
- Revenue by category (bar chart); review-score-vs-revenue ribbon chart by category
- Category summary table (avg. review score, item count, review count) and freight cost by category
- Slicers: seller state, product category

Logistics & Operation

- Actual vs. estimated delivery time trend (combo chart); order-status donut chart
- Delivery-time-vs-freight scatter plot by customer state
- KPI cards: AVG Delivery Days, On-Time Delivery Rate, Total Freight, Late Deliveries Count
- Delivery days by state (bar chart); slicers: seller city, product category

Customer Experience & Reviews

- Positive vs. negative review split (pie chart); review-score distribution (column chart)
- KPI cards: Avg Review Score, Total Reviews, On-Time Delivery Rate, Negative Review Rate
- Category-level table of review and on-time rates; comparison bar chart of negative/on-time/positive rates
- Slicer: product category

5. Tools & Technologies

Layer	Tool
Data storage & querying	SQL Server (T-SQL)
ETL / transformation	Power Query (M)
Data modeling	Power BI Desktop
Calculations	DAX
Reporting & visualization	Power BI Desktop

6. Data Quality Approach & Challenges

A recurring challenge in e-commerce order data is that logistics and payment fields are prone to silent errors — a missing delivery timestamp, a freight value of zero, or a payment total that doesn't reconcile with the order's line items. Rather than modeling the data first and discovering these issues inside a chart, this project ran a dedicated SQL audit pass before any table entered Power BI.

The audit checks (Section 4.1, Part 2) targeted four categories of risk: missing delivery dates on orders marked delivered, invalid price/freight values, payment-vs-order-value mismatches, and logically impossible date sequences. [Insert the specific record counts your queries returned here, if you'd like to cite them.]

This upstream validation step is what allows the KPIs on the Executive Overview page — average delivery days, on-time delivery rate, total revenue — to be presented with confidence rather than caveats.

7. Skills Demonstrated

- SQL (T-SQL): querying, aggregation, joins, view creation, data quality auditing
- Data modeling: relational schema design, star-schema principles in Power BI
- DAX: measure design across revenue, logistics, and satisfaction domains
- Report design: multi-page narrative structure, KPI selection, slicer/filter design
- Business analysis: translating stakeholder questions into report pages and metrics

8. Conclusion

The Olist E-Commerce Performance Dashboard demonstrates a complete analytics workflow — from raw, unvalidated relational data to a governed data model to a report built around specific business questions, rather than around whatever fields happened to be easiest to chart.

Possible next steps for extending this project include: connecting the model to a live/refreshing data source, adding a seller-performance page, or layering in a Python-based forecasting notebook on top of the same cleaned tables.

Appendix A — SQL Reference

Full contents of SQLQuery2.sql, included here for reference.

Part 1 — Basic Exploration

```
SELECT TOP 10 customer_id, customer_city, customer_state
FROM dbo.customers;

SELECT DISTINCT customer_state FROM dbo.customers ORDER BY customer_state ASC;

SELECT COUNT(order_id) AS Total_Orders FROM dbo.orders;

SELECT COUNT(order_id) AS Delivered_Orders FROM dbo.orders
WHERE order_status = 'delivered';

SELECT TOP 10 product_id, price FROM dbo.order_items ORDER BY price DESC;

SELECT payment_type, COUNT(order_id) AS Total_Transactions
FROM dbo.order_payments GROUP BY payment_type ORDER BY Total_Transactions DESC;

SELECT TOP 10 seller_city, COUNT(seller_id) AS Total_Sellers
FROM dbo.sellers GROUP BY seller_city ORDER BY Total_Sellers DESC;

SELECT review_score, COUNT(review_id) AS Total_Reviews
FROM dbo.order_reviews GROUP BY review_score ORDER BY review_score DESC;

SELECT MIN(price) AS Min_Price, MAX(price) AS Max_Price, AVG(price) AS Avg_Price,
      MIN(freight_value) AS Min_Freight, MAX(freight_value) AS Max_Freight
FROM dbo.order_items;

SELECT TOP 10 order_id, payment_type, payment_installments, payment_value
FROM dbo.order_payments ORDER BY payment_installments DESC;
```

Part 2 — Data Quality Checks

```
SELECT COUNT(order_id) AS Missing_Delivery_Dates FROM dbo.orders
WHERE order_status = 'delivered' AND order_delivered_customer_date IS NULL;

SELECT COUNT(order_id) AS Invalid_Price_Records FROM dbo.order_items
WHERE price <= 0 OR freight_value < 0;

SELECT TOP 10 i.order_id,
      ROUND(SUM(i.price + i.freight_value), 2) AS Items_Total,
      p.Payment_Total,
      ROUND(SUM(i.price + i.freight_value) - p.Payment_Total, 2) AS Difference
FROM dbo.order_items i
JOIN (SELECT order_id, ROUND(SUM(payment_value), 2) AS Payment_Total
      FROM dbo.order_payments GROUP BY order_id) p ON i.order_id = p.order_id
GROUP BY i.order_id, p.Payment_Total
HAVING ABS(SUM(i.price + i.freight_value) - p.Payment_Total) > 0.01
ORDER BY Difference DESC;

SELECT COUNT(order_id) AS Date_Error_Count FROM dbo.orders
WHERE order_delivered_customer_date < order_purchase_timestamp
      OR order_delivered_customer_date < order_delivered_carrier_date;

SELECT ROUND(AVG(DATEDIFF(DAY, order_purchase_timestamp, order_delivered_customer_date)), 1)
AS Avg_Delivery_Days,
      ROUND(AVG(DATEDIFF(DAY, order_estimated_delivery_date, order_delivered_customer_date)),
```

```

1) AS Avg_Delay_Vs_Estimate
FROM dbo.orders WHERE order_status = 'delivered';

SELECT TOP 10 t.product_category_name_english AS Category_Name,
              COUNT(oi.order_id) AS Total_Items_Sold, ROUND(SUM(oi.price), 2) AS Total_Sales
FROM dbo.order_items oi
JOIN dbo.products p ON oi.product_id = p.product_id
JOIN dbo.product_category_name_translation t ON p.product_category_name =
t.product_category_name
GROUP BY t.product_category_name_english ORDER BY Total_Sales DESC;

SELECT TOP 10 c.customer_state, COUNT(DISTINCT o.order_id) AS Total_Orders,
              ROUND(SUM(p.payment_value), 2) AS Total_Revenue
FROM dbo.orders o
JOIN dbo.customers c ON o.customer_id = c.customer_id
JOIN dbo.order_payments p ON o.order_id = p.order_id
WHERE o.order_status = 'delivered'
GROUP BY c.customer_state ORDER BY Total_Revenue DESC;

CREATE VIEW vw_orders_master AS
SELECT o.order_id, o.customer_id, o.order_status,
       o.order_purchase_timestamp, o.order_delivered_customer_date,
       o.order_estimated_delivery_date, c.customer_city, c.customer_state
FROM dbo.orders o JOIN dbo.customers c ON o.customer_id = c.customer_id;

```